# Chapter 19: Surrogate Optimization

Algorithms for Optimization, 2nd Edition (2026)

Kochenderfer & Wheeler

Lecture Notes

2026

# Chapter 19 Overview

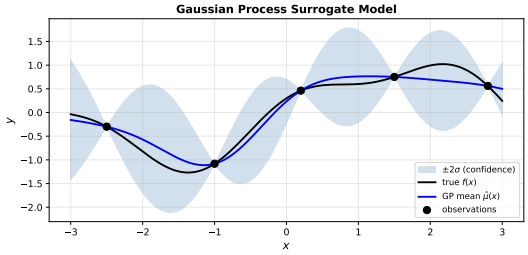
# Introduction: Why Surrogate Optimization?

## Motivation

- Many real-world objective functions $f : \mathbb{R}^n \to \mathbb{R}$ are **expensive** to evaluate (simulations, experiments).
- The previous chapter fitted a **Gaussian process (GP)** surrogate to infer Bayesian posterior predictive distributions over the true objective.
- Chapter 19 exploits these distributions to *guide where to sample next*.

## Core Idea

Use the GP surrogate to balance **exploitation** (sample near the predicted minimum) and **exploration** (sample where uncertainty is high).




GP surrogate $\hat{\mu}(x)$ with $\pm 2\sigma$ confidence band fitted to black observations.

# GP Surrogate: Key Quantities

At any unobserved point $\mathbf{x}$, the GP provides:

## Predictive Mean

$\hat{\mu}(\mathbf{x})$ — best estimate of $f(\mathbf{x})$

## Predictive Standard Deviation

$\hat{\sigma}(\mathbf{x})$ — uncertainty about $f(\mathbf{x})$

## Exploration Strategies Covered

1. Prediction-based exploration
2. Error-based exploration
3. Lower Confidence Bound (LCB/UCB)
4. Probability of Improvement (PoI)
5. Expected Improvement (EI)
6. Safe Optimization (SafeOpt)

**Bayesian framework:**

$$f(\mathbf{x}) \mid \text{data} \sim \mathcal{N}\big(\hat{\mu}(\mathbf{x}), \hat{\sigma}^2(\mathbf{x})\big)$$

- Large $\hat{\sigma} \Rightarrow$ high uncertainty $\Rightarrow$ explore.
- Small $\hat{\mu} \Rightarrow$ promising region $\Rightarrow$ exploit.

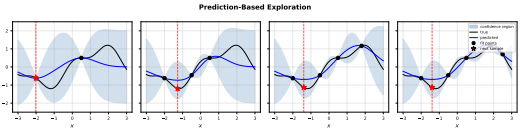
# 19.1 Prediction-Based Exploration

## Strategy

Select the next design point by **minimizing the predicted mean** of the surrogate:

$$\mathbf{x}^{(m+1)} = \arg\min_{\mathbf{x}} \hat{\mu}(\mathbf{x}) \tag{19.1}$$

- Pure **exploitation**: always samples where GP thinks the minimum is.
- Simple, but can get **stuck** at a single point if the GP mean has a single global minimizer.
- **Failure mode:** With a zero-mean GP and a single sample $\mathbf{x}^{(1)}$, the predicted mean has a single global minimizer at $\mathbf{x}^{(1)}$ — the method will keep sampling the same point.




Four iterations of prediction-based exploration. Red star = next sample = $\arg\min \hat{\mu}$.

# Prediction-Based Exploration: Python Implementation

```python
import numpy as np
from scipy.optimize import minimize
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel

def prediction_based_optimization(f, bounds, n_init=3, n_iter=10):
    """
    Prediction-based (pure exploitation) surrogate optimization.
    f      : expensive objective function  f: R^n -> R
    bounds : list of (lo, hi) per dimension
    """
    dim = len(bounds)
    # Initial random samples
    lo = np.array([b[0] for b in bounds])
    hi = np.array([b[1] for b in bounds])
    X = lo + np.random.rand(n_init, dim) * (hi - lo)
    y = np.array([f(x) for x in X])

    kernel = ConstantKernel(1.0) * RBF(length_scale=1.0)
    gp = GaussianProcessRegressor(kernel=kernel, n_restarts_optimizer=5,
                                  normalize_y=True)
    for _ in range(n_iter):
        gp.fit(X, y)

        # Minimize predicted mean  (equation 19.1)
        best = None
        for x0 in X[np.argsort(y)[:3]]:  # warm start from best known
```

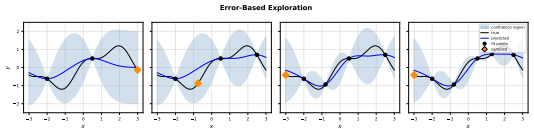
# 19.2 Error-Based Exploration

## Strategy

Select the next design point where the GP has the **highest uncertainty** (maximum standard deviation):

$$\mathbf{x}^{(m+1)} = \arg\max_{\mathbf{x}} \hat{\sigma}(\mathbf{x}) \tag{19.2}$$

- Pure **exploration**: fills in gaps in the input space.
- Ignores the objective function values — does *not* try to minimize $f$.
- **Problem:** GP with unbounded domain always has high uncertainty far from samples, pulling exploration away from the region of interest.
- **Fix:** Constrain exploration to a *closed region*.




Orange diamond = next sample = $\arg\max \hat{\sigma}$.
Samples fill the input space uniformly.

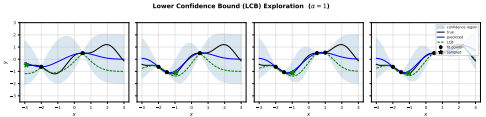
# 19.3 Lower Confidence Bound (LCB) Exploration

## The LCB Acquisition Function

Combine prediction and uncertainty by sampling the point that minimizes the **lower confidence bound**:

$$\mathbf{x}^{(m+1)} = \arg\min_{\mathbf{x}} \left[\hat{\mu}(\mathbf{x}) - \alpha\, \hat{\sigma}(\mathbf{x})\right] \tag{19.3}$$

where $\alpha > 0$ is a **trade-off scalar**.




Green dashed = LCB= $\hat{\mu} - \alpha\hat{\sigma}$. Green star = $\arg\min$ LCB.

**Symbol glossary:**

- $\hat{\mu}(\mathbf{x})$ — GP predicted mean at $\mathbf{x}$
- $\hat{\sigma}(\mathbf{x})$ — GP predicted std. deviation at $\mathbf{x}$
- $\alpha$ — exploration weight ($\alpha \uparrow \Rightarrow$ more exploration)

## Interpretation

- $\alpha = 0$: reduces to prediction-based (pure exploitation).

# LCB Exploration: Python

```python
import numpy as np
from scipy.optimize import differential_evolution
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel

def lcb_optimization(f, bounds, alpha=1.0, n_init=3, n_iter=15):
    """
    Bayesian optimization with Lower Confidence Bound acquisition.
    Parameters
    ----------
    alpha : float
        Exploration weight. Higher alpha => more exploration.
    """
    dim = len(bounds)
    lo = np.array([b[0] for b in bounds])
    hi = np.array([b[1] for b in bounds])
    X = lo + np.random.rand(n_init, dim) * (hi - lo)
    y = np.array([f(x) for x in X])

    kernel = ConstantKernel(1.0) * RBF(length_scale=1.0)
    gp = GaussianProcessRegressor(kernel=kernel, n_restarts_optimizer=5,
                                  normalize_y=True)
    for k in range(n_iter):
        gp.fit(X, y)

        # LCB acquisition: minimize  mu - alpha * sigma  (Eq. 19.3)
        def lcb(x):
```

# 19.4 Probability of Improvement (PoI)

## Improvement Function

Define the improvement at a point **x** as:

$$I(y) = \max(y_{\min} - y,\ 0) \tag{19.4}$$

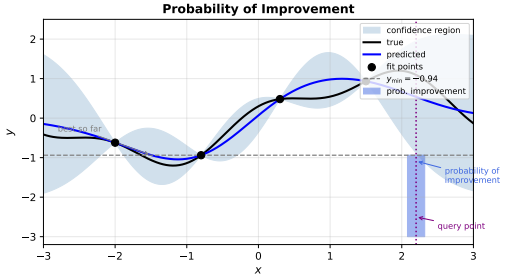
where $y_{\min}$ is the **best value sampled so far**.

## Probability of Improvement (PoI)

For $\hat{\sigma} > 0$, the PoI is:

$$P(y < y_{\min}) = \int_{-\infty}^{y_{\min}} \mathcal{N}(y \mid \hat{\mu}, \hat{\sigma}^2)\, dy \tag{19.5}$$

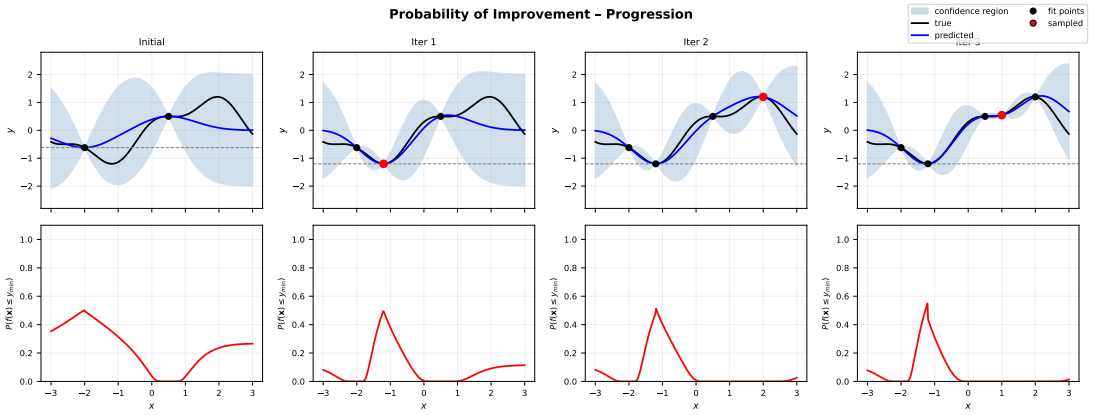
$$= \Phi\left(\frac{y_{\min} - \hat{\mu}}{\hat{\sigma}}\right) \tag{19.6}$$




Shaded blue region below $y_{\min}$ at the query point is the probability of improvement.

**Symbol glossary:**

# PoI: Progression Across Iterations




- **Top row:** GP posterior with evolving fit points (black dots, red = newly sampled).
- **Bottom row:** Probability of improvement $P(f(\mathbf{x}) \leq y_{\min})$.
- PoI concentrates probability mass at points likely to beat the current best.
- **Limitation:** Maximizing PoI does not care *how much* improvement occurs — it tends to sample points barely better than $y_{\min}$.

# PoI: Python Implementation (Algorithm 19.1)

```python
import numpy as np
from scipy.stats import norm
from scipy.optimize import differential_evolution
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel

def prob_of_improvement(y_min, mu, sigma):
    """
    Compute PoI = Phi((y_min - mu) / sigma).
    Returns 0 where sigma == 0 (noiseless observations).
    Implements Eq. (19.6).
    """
    with np.errstate(divide='ignore', invalid='ignore'):
        z = np.where(sigma > 0, (y_min - mu) / sigma, 0.0)
    return np.where(sigma > 0, norm.cdf(z), 0.0)

def poi_optimization(f, bounds, n_init=3, n_iter=15):
    """Bayesian optimization with Probability of Improvement acquisition."""
    dim = len(bounds)
    lo = np.array([b[0] for b in bounds])
    hi = np.array([b[1] for b in bounds])
    X = lo + np.random.rand(n_init, dim) * (hi - lo)
    y = np.array([f(x) for x in X])

    kernel = ConstantKernel(1.0) * RBF(length_scale=1.0)
    gp = GaussianProcessRegressor(kernel=kernel, n_restarts_optimizer=5,
                                  normalize_y=True)
```

# 19.5 Expected Improvement (EI) — Derivation

## EI: Motivation

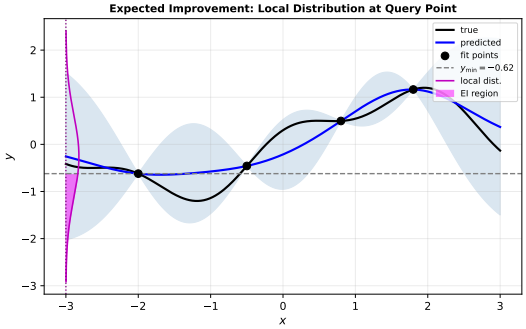
PoI ignores the *magnitude* of improvement.
**Expected Improvement** averages the improvement over the predictive distribution.

**Substitution:** Let $z = (y - \hat{\mu})/\hat{\sigma}$, $y'_{\min} = (y_{\min} - \hat{\mu})/\hat{\sigma}$. The improvement becomes:

$$I(y) = \begin{cases} \hat{\sigma}(y'_{\min} - z) & \text{if } z < y'_{\min} \\ 0 & \text{otherwise} \end{cases} \tag{19.8}$$

## Closed-Form EI (Eq. 19.12)

$$\mathbb{E}[I(y)] = (y_{\min} - \hat{\mu})\,\Phi\left(\frac{y_{\min} - \hat{\mu}}{\hat{\sigma}}\right) + \hat{\sigma}^2\,\mathcal{N}(y_{\min} \mid \hat{\mu}, \hat{\sigma}^2)$$




Magenta region: the portion of the local normal distribution that falls below $y_{\min}$ (the EI integrand).

# EI: Full Derivation Steps

Starting from $\mathbb{E}[I(y)]$ (Eq. 19.9):

$$\mathbb{E}[I(y)] = \hat{\sigma} \int_{-\infty}^{y'_{\min}} (y'_{\min} - z)\, \mathcal{N}(z \mid 0, 1)\, dz \tag{19.9}$$

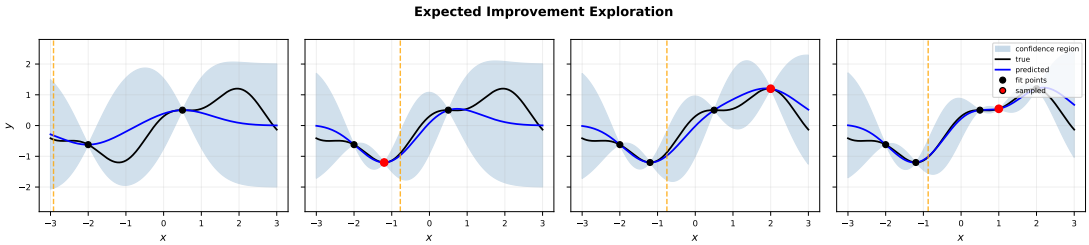
$$= \hat{\sigma} \left[ y'_{\min} \int_{-\infty}^{y'_{\min}} \mathcal{N}(z|0,1) dz - \int_{-\infty}^{y'_{\min}} z\, \mathcal{N}(z|0,1) dz \right] \tag{19.10}$$

$$= \hat{\sigma} \left[ y'_{\min}\, P(z \le y'_{\min}) + \mathcal{N}(y'_{\min}|0,1) - \underbrace{\mathcal{N}(-\infty|0,1)}_{=0} \right] \tag{19.11}$$

$$\boxed{\mathbb{E}[I(y)] = (y_{\min} - \hat{\mu})\, \Phi\!\left( \frac{y_{\min} - \hat{\mu}}{\hat{\sigma}} \right) + \hat{\sigma}^2\, \mathcal{N}\big(y_{\min} \,|\, \hat{\mu}, \hat{\sigma}^2\big)} \tag{19.12}$$

- When $\hat{\sigma} = 0$ (noise-free observed point): $\mathbb{E}[I] = 0$.
- $\mathbb{E}[I] > 0$ everywhere with $\hat{\sigma} > 0$, giving a smooth acquisition landscape.
- EI **automatically balances** exploitation (large $y_{\min} - \hat{\mu}$) and exploration (large $\hat{\sigma}$).

# EI: Progression Across Iterations




- Orange dashed line $= \arg\max_{\mathbf{x}} \mathbb{E}[I(\mathbf{x})]$ — next query point.
- Red dots = most recently sampled points.
- EI steers towards **both** low predicted values *and* high uncertainty, avoiding the "barely improving" issue of PoI.
- **Complexity per iteration:** $\mathcal{O}(m^3)$ for GP fitting ($m$ observations); $\mathcal{O}(m^2)$ for prediction at $n$ test points.

# Expected Improvement: Python (Algorithm 19.2)

```python
import numpy as np
from scipy.stats import norm
from scipy.optimize import differential_evolution
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel

def expected_improvement(y_min, mu, sigma):
    """
    Closed-form Expected Improvement (Eq. 19.12).
    Returns EI = 0 where sigma == 0.
    """
    with np.errstate(divide='ignore', invalid='ignore'):
        z = np.where(sigma > 0, (y_min - mu) / sigma, 0.0)
    ei = np.where(
        sigma > 0,
        (y_min - mu) * norm.cdf(z) + sigma * norm.pdf(z),
        0.0
    )
    return np.maximum(ei, 0.0)

def ei_optimization(f, bounds, n_init=3, n_iter=15):
    """Bayesian optimization with Expected Improvement acquisition."""
    dim = len(bounds)
    lo = np.array([b[0] for b in bounds]); hi = np.array([b[1] for b in bounds])
    X = lo + np.random.rand(n_init, dim) * (hi - lo)
    y = np.array([f(x) for x in X])
```

# Comparing Acquisition Functions

## Summary Table

| Method | Exploits | Explores |
|---|---|---|
| Prediction-based | ✓ | × |
| Error-based | × | ✓ |
| LCB ($\alpha$) | ✓ | ✓ |
| PoI | ✓ | limited |
| EI | ✓ | ✓ |

## Numerical Running Example

$f(x) = \sin(x) + 0.3\cos(3x)$, $x \in [-3, 3]$

Samples at $x \in \{-2, 0.5\}$, $y_{\min} = -0.918$:

| Method | Next $x$ | Value |
|---|---|---|
| Pred.-based | $-0.82$ | $-1.30$ |
| Error-based | $-3.0$ | (boundary) |
| LCB ($\alpha = 1$) | $-1.5$ | $-1.10$ |
| PoI | $-1.2$ | $-1.08$ |
| EI | $-0.9$ | $-1.28$ |

## Key Insight

LCB, PoI, and EI all require tuning or have different biases. **EI** is the most popular for unconstrained problems because it adapts automatically.

# 19.6 Safe Optimization (SafeOpt) — Motivation

## Problem Setting

Some applications **cannot evaluate unsafe designs**:

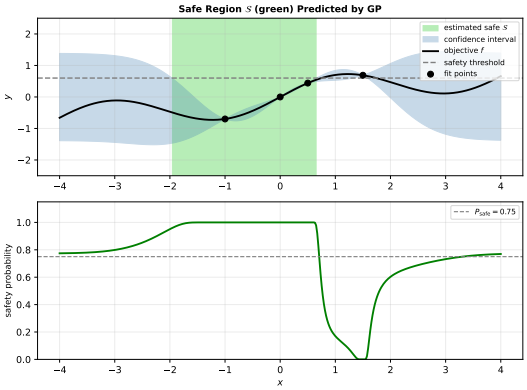
- Medical device tuning (patient safety)
- Robotics (hardware damage)
- Chemical processes (hazard)

We need an optimization strategy that *never* queries a point believed to be unsafe.

## Safety Constraint

A design $\mathbf{x}$ is **safe** if $f(\mathbf{x}) \leq y_{\max}$ (a known safety threshold).

- Gaussian process confidence intervals quantify whether a point is *likely* safe.
- SafeOpt operates only within the **estimated**




Top: green = estimated safe region $\mathcal{S}$. Bottom: safety probability with threshold $P_{\text{safe}}$.

# SafeOpt: Confidence Bounds and Sets

## Confidence Bounds (Algorithm 19.4)

For every design point $\mathbf{x} \in \mathcal{X}$, maintain:

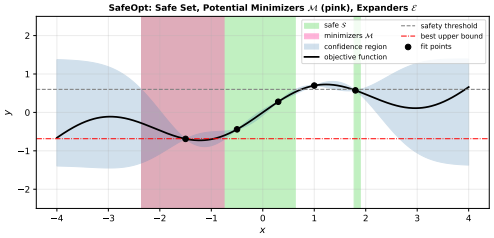
$$u(\mathbf{x}) = \hat{\mu}(\mathbf{x}) + \sqrt{\beta}\,\hat{\sigma}(\mathbf{x}) \qquad \text{(upper bound)}$$

$$\ell(\mathbf{x}) = \hat{\mu}(\mathbf{x}) - \sqrt{\beta}\,\hat{\sigma}(\mathbf{x}) \qquad \text{(lower bound)}$$

Width: $w_i(\mathbf{x}) = u(\mathbf{x}) - \ell(\mathbf{x})$

## Key Sets

- **Safe set** $\mathcal{S}_i$: $\{x \mid u(x) \leq y_{\max}\}$ (green)
- **Minimizers** $\mathcal{M}_i$: safe points whose *lower* bound is below the best *upper* bound in $\mathcal{S}_i$ (pink)
- **Expanders** $\mathcal{E}_i$: safe points that, if added, could enlarge $\mathcal{S}$ (orange)




Green = safe $\mathcal{S}$, pink = minimizers $\mathcal{M}$, red dashed = best upper bound.

# SafeOpt: Formal Definitions

## Potential Minimizers $\mathcal{M}_i$ (Algorithm 19.5)

$$\mathcal{M}_i = \left\{ \mathbf{x} \in \mathcal{S}_i \mid \ell_i(\mathbf{x}) < \min_{\mathbf{x}' \in \mathcal{S}_i} u_i(\mathbf{x}') \right\} \qquad (19.13 - 19.14)$$

Points whose lower confidence bound is below the best safe upper bound. *Cannot rule out being the global minimizer within $\mathcal{S}_i$.*

## Potential Expanders $\mathcal{E}_i$

Points in $\mathcal{S}_i$ that, optimistically (assuming $f = \ell$), could add new points to $\mathcal{S}$:

$$\mathcal{E}_i = \left\{ \mathbf{x} \in \mathcal{S}_i \mid \exists \mathbf{x}' \notin \mathcal{S}_i : \ \ell_i(\mathbf{x}') \leq y_{\max} \right\} \qquad (19.15 - 19.16)$$

**Naturally lie near the boundary** of the safe region.

## Query Selection (Eq. 19.17)

$$\mathbf{x}^{(i)} = \arg \max_{\mathbf{x} \in \mathcal{M}_i \cup \mathcal{E}_i} w_i(\mathbf{x}) \quad \text{where } w_i(\mathbf{x}) = u_i(\mathbf{x}) - \ell_i(\mathbf{x}) \qquad (19.17)$$

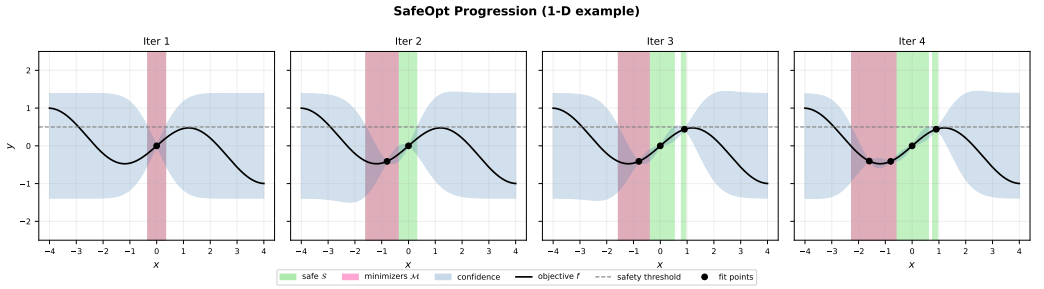
# SafeOpt Algorithms (Python)

```python
import numpy as np
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel

def update_confidence_intervals(gp, X_grid, beta=3.0):
    """Algorithm 19.4: upper/lower bounds at all grid points."""
    mu, sigma = gp.predict(X_grid, return_std=True)
    u = mu + np.sqrt(beta) * sigma    # upper confidence bound
    l = mu - np.sqrt(beta) * sigma    # lower confidence bound
    return u, l

def compute_sets(gp, X_grid, u, l, y_max, beta=3.0):
    """Algorithm 19.5: safe S, minimizers M, expanders E."""
    S = u <= y_max                                  # safe set
    M = np.zeros(len(X_grid), dtype=bool)
    E = np.zeros(len(X_grid), dtype=bool)
    if np.any(S):
        best_u = np.min(u[S])                       # best safe upper bound
        M[S] = l[S] < best_u                        # potential minimizers
        w_max = np.max(u[M] - l[M]) if np.any(M) else 0.0
        # Expanders: safe, not minimizer, and optimistic expansion possible
        cands = S & ~M
        for i in np.where(cands)[0]:
            if u[i] - l[i] > w_max:
                E[i] = True
    return S, M, E
```

# SafeOpt: 1-D Progression




- Algorithm starts from a **single known safe point**.
- Green region expands as the GP becomes more confident.
- Pink region (minimizers $\mathcal{M}$) narrows in on the local safe minimum.

## Key Limitation

SafeOpt can **never reach** a global optimum that lies across an unsafe region — it is limited to the *locally reachable* safe region.

# SafeOpt: Algorithm Summary

## SafeOpt (Algorithm 19.3) — Step-by-Step

1. Initialize with $\mathbf{x}^{(1)}$ known safe. Push $(\mathbf{x}^{(1)}, f(\mathbf{x}^{(1)}))$ to the GP.
2. **For** $k = 1, \ldots, k_{\max}$:
   1. Update confidence bounds $u_k, \ell_k$ for all $\mathbf{x} \in \mathcal{X}$ (Algorithm 19.4).
   2. Compute sets $\mathcal{S}_k$, $\mathcal{M}_k$, $\mathcal{E}_k$ (Algorithm 19.5).
   3. If $\mathcal{M}_k \cup \mathcal{E}_k = \emptyset$: **break**.
   4. Query $\mathbf{x}^{(k)} = \arg\max_{\mathbf{x} \in \mathcal{M}_k \cup \mathcal{E}_k} w_k(\mathbf{x})$ (Algorithm 19.6).
   5. Evaluate $f(\mathbf{x}^{(k)})$ and update GP.
3. Return best safe upper bound and its index in $\mathcal{X}$.

## Complexity

- GP update: $\mathcal{O}(m^3)$ per iteration
- Set computation: $\mathcal{O}(|\mathcal{X}| \cdot m^2)$
- Total: $\mathcal{O}(k_{\max} \cdot |\mathcal{X}| \cdot m^2)$

## Parameters

- $\beta \geq 0$ — confidence scalar (default 3.0)
- $y_{\max}$ — safety threshold
- $k_{\max}$ — maximum iterations
- $\mathcal{X}$ — finite design space (grid)

# SafeOpt: Python Numerical Example

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

# Objective: f(x) = 0.6*sin(1.2x) - 0.1x    on [-4, 4]
# Safety threshold: y_max = 0.5
# Initial safe point: x = 0.0   (f(0) = 0.0 <= 0.5)

def f_safe(x):
    return 0.6 * np.sin(1.2 * x) - 0.1 * x

X_grid = np.linspace(-4, 4, 100).reshape(-1, 1)
y_max  = 0.5
i_safe = np.argmin(np.abs(X_grid[:, 0] - 0.0))  # index of x=0

x_best, u_best = safe_opt(
    f     = lambda x: f_safe(x[0]),
    X_grid = X_grid,
    i_safe = i_safe,
    y_max  = y_max,
    beta   = 3.0,
    k_max  = 20
)

if x_best is not None:
    print(f"SafeOpt best safe point: x = {x_best[0]:.4f}")
```

# 19.7 Summary

## Key Takeaways

- **Prediction-based**: purely exploits the GP mean; can stagnate at a single point.
- **Error-based**: purely explores; ignores the objective.
- **LCB**: tunable balance via $\alpha$; simple and effective.
- **PoI**: maximizes probability of beating $y_{\min}$; biased toward marginal improvements.
- **EI**: expected magnitude of improvement; most widely used acquisition function; parameter-free.
- **SafeOpt**: respects safety constraints via GP confidence bounds; provably safe under GP assumptions; limited to locally reachable safe regions.

## Common Thread

All methods use the GP posterior to balance **exploration** (reduce uncertainty) and **exploitation** (improve the objective).

## When to Use Which?

| Scenario | Recommended |
|---|---|
| Unconstrained, general | EI |
| Safety-critical | SafeOpt |
| Fast/simple | LCB |
| High-dim exploration | Error-based |

# 19.8 Exercises — Exercise 19.1 & 19.2

## Exercise 19.1

**Give an example where prediction-based optimization fails.**

## Solution 19.1

Suppose we use a zero-mean GP and start with a single observation $(\mathbf{x}^{(1)}, y^{(1)})$. The predicted mean has a single global minimizer at $\mathbf{x}^{(1)}$. Prediction-based optimization will **repeatedly sample** $\mathbf{x}^{(1)}$ and never explore elsewhere.

## Exercise 19.2

**Main difference between LCB and error-based exploration?**

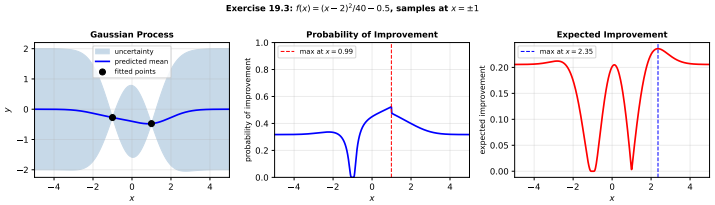
## Solution 19.2

- **Error-based** exploration only reduces variance; it wastes effort exploring regions with high uncertainty but does *not* actively seek to minimize $f$.
- **LCB** combines minimizing the predicted mean *and* reducing uncertainty via the $-\alpha\hat{\sigma}$ term, making it an effective optimization strategy.

# 19.8 Exercise 19.3 — Numerical Example

## Problem Statement

$f(x) = (x-2)^2/40 - 0.5$ on $[-5, 5]$, with squared-exponential kernel $k(x, x') = \exp(-r^2/2)$ and zero mean. Evaluation points at $x = -1$ and $x = 1$. Find the next query point under (a) maximum PoI and (b) maximum EI.

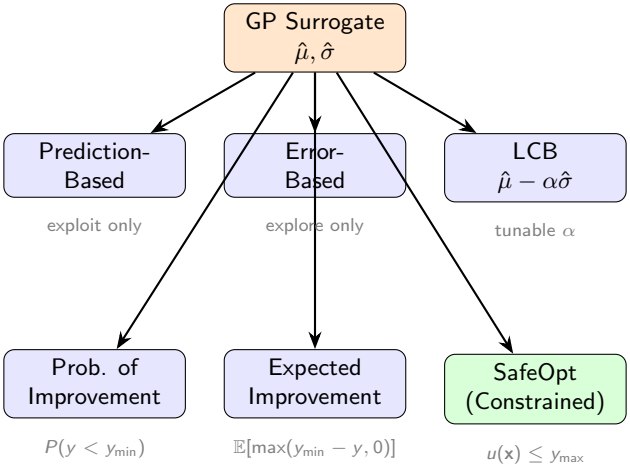



Left: GP posterior. Centre: PoI. Right: EI.

## Solution

- $y_{\min} = f(-1) = \min\{f(-1), f(1)\}$
  $f(-1) = \frac{9}{40} - 0.5 = -0.275$
  $f(1) = \frac{1}{40} - 0.5 = -0.475$
  $\Rightarrow y_{\min} = -0.475$
- **Max PoI:** $x \approx 0.84$
  (just past $x = 1$, barely improving)
- **Max EI:** $x \approx 2.78$
  (close to true min at $x = 2$,

# Chapter 19 — Concept Map



# References and Further Reading


- Kochenderfer & Wheeler, *Algorithms for Optimization*, 2nd ed., MIT Press, 2026. Chapter 19.
- Srinivas, N., Krause, A., Kakade, S., & Seeger, M. (2010). *Gaussian process optimization in the bandit setting*. ICML.
- Berkenkamp, F., Krause, A., & Schoellig, A. P. (2016). *Safe controller optimization for quadrotors with Gaussian processes*. ICRA.
- Berkenkamp, F., Turchetta, M., Schoellig, A. P., & Krause, A. (2017). *Safe model-based reinforcement learning with stability guarantees*. NeurIPS.
- Jones, D. R., Schonlau, M., & Welch, W. J. (1998). *Efficient global optimization of expensive black-box functions*. Journal of Global Optimization.